

ISL Technical Report-Team 1

Title: Predictive Analysis on Supervised and Unsupervised Methods.

Team-1:

Praneet Kasinaduni

Data Science

UMKC

16354683

pk65d@umsystem.edu

Sindhu Mukkara

Data Science

UMKC

16359054

smpyg@umsystems.edu

Tejeswar Reddy Narreddy

Data Science

UMKC

1 6353073

tnppc@umsystem.edu

Vinay Joshva

Data Science

UMKC

16352584

vjmh9@umsystem.edu

COMP-SCI 5565

Introduction to Statistical Learning

Prof. Adu Baffour

Supervised and unsupervised methods are two broad categories of machine learning techniques that are used to extract patterns and insights from data. The distinction between them lies in whether the data used for training has labeled or unlabeled examples.

Supervised Methods:

Supervised learning techniques involve training models on labeled datasets, where the desired output or target variable is known for each input instance. The primary objective is to develop a model that can accurately predict the corresponding outputs or labels for new, unseen inputs. Some widely-used supervised learning algorithms are:

Linear and Logistic Regression:

These algorithms are employed for regression and classification problems, respectively. In regression tasks, the goal is to predict a continuous target variable based on the input features. In classification tasks, the aim is to assign categorical labels to the input instances.

Decision Trees and Random Forests:

Decision trees are tree-like models that recursively partition the input space based on the input features, creating a hierarchical structure for decision-making. Random forests are ensemble methods that combine multiple decision trees, improving predictive performance and mitigating overfitting issues.

Support Vector Machines (SVMs):

SVMs are powerful algorithms that construct high-dimensional hyperplanes to separate data points into different classes or categories. These algorithms are particularly effective in handling complex, non-linear decision boundaries and are widely used for classification tasks.

In essence, supervised learning methods leverage labeled data to learn the underlying patterns and relationships between the input features and the target variable. The trained model can then be used to make predictions or classifications for new, unseen data instances based on the learned patterns.

Decision Trees:

Decision trees are a widely adopted supervised learning technique that can handle both classification and regression problems. They operate by recursively partitioning the input feature space into smaller subsets based on the values of the features themselves. This divide-and-conquer approach results in a tree-like structure that provides an interpretable and transparent representation of the decision-making process. Decision trees are particularly valuable in scenarios where stakeholders require an understanding and trust in the decision-making process, as their inherent interpretability and comprehensibility make it easier to explain the reasoning behind the decisions made by the model.

Linear Discriminant Analysis:

Linear Discriminant Analysis (LDA) is a supervised learning method that combines dimensionality reduction and classification capabilities. Its primary goal is to identify linear combinations of features that effectively separate the different classes or categories present in the dataset. When the underlying assumptions of LDA are met, such as normally distributed data and equal covariance matrices across classes, it can be an efficient technique for classification tasks and dimensionality reduction. LDA aids in the accurate classification of new instances by projecting them onto the discriminant subspace, while simultaneously providing insights into the structure of the data by identifying the linear combinations of features that maximize class separability.

Unsupervised Methods:

In contrast to supervised methods, unsupervised learning techniques operate on unlabeled data, where the objective is to uncover inherent patterns, structures, or relationships within the data without relying on predefined labels or target outputs. Some widely adopted unsupervised learning algorithms include:

Clustering Algorithms:

Clustering algorithms aim to group similar data points together based on their features or characteristics, forming distinct clusters. Notable examples of clustering techniques include K-Means Clustering, Hierarchical Clustering, and Density-Based Spatial Clustering of Applications with Noise (DBSCAN).

Dimensionality Reduction Techniques:

These methods focus on reducing the number of features or dimensions in the data while preserving the most informative and relevant information. Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) are popular dimensionality reduction techniques that can aid in data visualization, feature selection, and curse of dimensionality mitigation.

Association Rule Mining:

Association rule mining algorithms are employed to discover frequent patterns or associations between items in large datasets. This technique finds applications in areas such as market basket analysis in retail, where it can identify products that are frequently purchased together, enabling targeted marketing strategies and product recommendations.

Unsupervised learning techniques find their strength in exploratory data analysis scenarios, where the primary objective is to unveil hidden structures, patterns, or relationships within the data without relying on labeled examples or predefined outputs.

Anomaly Detection Algorithms:

These algorithms are designed to identify rare or unusual data points that deviate significantly from the majority of the data. Anomaly detection techniques can prove invaluable in detecting fraudulent activities, system failures, or identifying outliers that may warrant further investigation.

The selection between supervised and unsupervised learning methods is contingent upon the nature of the problem at hand and the availability of labeled data. Supervised learning techniques are typically employed when the goal is to make predictions or classify instances based on labeled training examples. Conversely, unsupervised learning methods are preferred when the objective is to uncover underlying patterns, structures, or relationships within the data without the presence of explicit labels.

In practical scenarios, the choice between supervised and unsupervised approaches is often dictated by the specific problem domain, the characteristics of the available data, and the desired outcomes. Supervised learning excels in tasks where labeled data is accessible and the goal is to make accurate predictions or classifications. Unsupervised learning, on the other hand, shines in situations where the data is unlabeled, and the aim is to uncover hidden insights, group similar instances, or identify anomalies.

K-Means Clustering:

A popular unsupervised machine learning technique called K-means clustering divides a given dataset into K different groups or clusters according to similarity or distance metrics. Provides data points to one of the K clusters based on the distance from the cluster center. Initially, a cluster centroid is chosen at random throughout the space. Subsequently, the distance between each data point and the cluster centroid determines which cluster it belongs to. Once every point has been assigned to a

cluster, new cluster centroids are assigned. This process keeps going gradually until it locates a good cluster.

Hierarchical Clustering:

An unsupervised machine learning technique called hierarchical clustering is used to arrange data points into a hierarchy of groups. Unlike K-Means clustering, which needs this information, hierarchical clustering does not have a set number of clusters. Instead, it clusters the data points at various levels of the hierarchy to create a dendrogram, or tree-like structure, which represents the granularity.

In actuality, a lot of real-world issues could combine supervised and unsupervised approaches. For example, unsupervised techniques might be used to extract features from data or prepare it for processing before supervised algorithms are applied for tasks like classification or prediction.

ISLP package:

The module ISLP (Introduction to Statistical Learning with Python) and all of its dependencies are being installed. Several statistical learning methods, including support for generalised additive models, survival analysis, and PyTorch integration, are included in the ISLP package. NumPy, SciPy, pandas, scikit-learn, and other relevant libraries must be downloaded and installed in order to complete the installation procedure. We were able to utilize ISLP for statistical learning tasks in Python since the procedure is successful overall.

IMPORTED LIBRARIES:

1. NumPy (import numpy as np):

NumPy is a fundamental package for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with mathematical functions to operate on these arrays.

2. Matplotlib (from matplotlib.pyplot import subplots, cm):

Matplotlib is a 2D plotting library for Python. subplots is a function for creating a figure and a set of subplots, while cm stands for colormap and is used for mapping data values to colors in a plot.

3. from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

Classification technique used in machine learning, It seeks to find the linear combinations of features that best separate multiple classes in the dataset while preserving the class-discriminatory information as much as possible.

4. ISLP (from ISLP import load_data, confusion_table):

ISLP is a package, and here it's used to import two specific functionalities: 'load_data' for loading datasets, and 'confusion_table' for creating a confusion table for classification model evaluation.

5. StandardScaler from scikit-learn (from sklearn.preprocessing import StandardScaler):

'StandardScaler' is used for standardizing features by removing the mean and scaling to unit variance, a common preprocessing step in machine learning.

6. accuracy_score from scikit-learn (from sklearn.metrics import accuracy_score):

'accuracy_score' is a function to calculate the accuracy of a classification model.

7. train_test_split from scikit-learn (from sklearn.model_selection import train_test_split):

'train_test_split' is a function for splitting datasets into training and testing sets, essential for model evaluation.

8. **from sklearn.metrics import confusion_matrix:**

'confusion_matrix' is a function used to compute the confusion matrix, which is a table that describes the performance of a classification model.

9. **from sklearn.metrics import silhouette_score:**

The silhouette score is a metric used to evaluate the quality of clusters created by clustering algorithms, such as K-Means clustering or hierarchical clustering. It measures how similar an object is to its own cluster compared to other clusters. A higher silhouette score indicates that the object is well-matched to its own cluster and poorly matched to neighbouring clusters.

10. **from scipy.cluster.hierarchy import dendrogram, linkage:**

The dendrogram function is used to visualize the hierarchical clustering dendrogram, which represents the hierarchical structure of clusters as a tree-like diagram. It takes as input the result of hierarchical clustering and plots the dendrogram using matplotlib.

11. **from scipy.cluster.hierarchy import fcluster:**

The 'fcluster' function is used to assign cluster labels to observations based on a hierarchical clustering result. It allows us to specify a threshold or the number of clusters, and it assigns each observation to a cluster based on the hierarchical clustering structure.

12. **from scipy.stats import norm:**

The norm class in SciPy is used to represent a normal (Gaussian) distribution. It provides various methods and properties for working with normal distributions, such as computing probability density function (PDF), cumulative distribution function (CDF), generating random samples, and fitting a normal distribution to data.

Data Description:

✓ Data Description

✓
0s

```
# Display the list of features (columns)
features = dryBeans_dataset.columns.tolist()
print("Features (Columns):")
print(features)

# Display the number of samples (rows)
num_samples = len(dryBeans_dataset)
print("\nNumber of Samples (Rows):", num_samples)

print(dryBeans_dataset.info())
```

Firstly, we're writing to display the list of features. Features variable retrieves the list of column names using 'columns.tolist()' method. It prints list of features to the console. Using the len() function, we're calculating the number of samples i.e. the rows present in the dataframe. Num_samples print the number of samples to the console. Last statement 'dryBeans_dataset.info()' method gives us a brief summary of the complete dataset. The below image shows the output acquired by executing the above programs.

```

Features (Columns):
['Area', 'Perimeter', 'MajorAxisLength', 'MinorAxisLength', 'AspectRatio', 'Eccentricity', 'ConvexArea', 'EquivDiameter', 'Extent', 'Solidity', 'roundness',

Number of Samples (Rows): 13611
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13611 entries, 0 to 13610
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Area                  13611 non-null  int64
1   Perimeter             13611 non-null  float64
2   MajorAxisLength       13611 non-null  float64
3   MinorAxisLength       13609 non-null  float64
4   AspectRatio           13608 non-null  float64
5   Eccentricity           13611 non-null  float64
6   ConvexArea            13609 non-null  float64
7   EquivDiameter         13610 non-null  float64
8   Extent                13608 non-null  float64
9   Solidity              13608 non-null  float64
10  roundness             13611 non-null  float64
11  Compactness           13611 non-null  float64
12  ShapeFactor1          13610 non-null  float64
13  ShapeFactor2          13610 non-null  float64
14  ShapeFactor3          13610 non-null  float64
15  ShapeFactor4          13611 non-null  float64
16  Class                 13611 non-null  object
dtypes: float64(15), int64(1), object(1)
memory usage: 1.8+ MB
None

```

Data preprocessing:

The data preprocessing involves checking our dataset and rectifying the dataset from any existing null values which might hinder our further exploration of the dataset. The code below shows usage of 'isnull().sum()' method to count the number of null values present in the features of our dry beans dataset. It is evident that there are not many null values, but some null values are present.

```

# Check for null values
null_values = dryBeans_dataset.isnull().sum()
print("Null Values:\n", null_values)

```

```

Null Values:
Area                0
Perimeter           0
MajorAxisLength     0
MinorAxisLength     2
AspectRatio          3
Eccentricity         0
ConvexArea          2
EquivDiameter       1
Extent              3
Solidity             3
roundness           0
Compactness         0
ShapeFactor1        1
ShapeFactor2        1
ShapeFactor3        1
ShapeFactor4        0
Class               0
dtype: int64

```

The code below shows how we are handling the null values in our dataset. We are simply performing custom imputation based on each column name. This ensures that it fills a dummy value in place of null values present in the features using 'fillna()' method.

```
#handling null values
# Custom imputation based on domain knowledge
dryBeans_dataset['MinorAxisLength'].fillna(value=0, inplace=True)
dryBeans_dataset['AspectRatio'].fillna(value=0, inplace=True)
dryBeans_dataset['ConvexArea'].fillna(value=0, inplace=True)
dryBeans_dataset['EquivDiameter'].fillna(value=0, inplace=True)
dryBeans_dataset['Extent'].fillna(value=0, inplace=True)
dryBeans_dataset['Solidity'].fillna(value=0, inplace=True)
dryBeans_dataset['ShapeFactor1'].fillna(value=0, inplace=True)
dryBeans_dataset['ShapeFactor2'].fillna(value=0, inplace=True)
dryBeans_dataset['ShapeFactor3'].fillna(value=0, inplace=True)
```

Upon checking for null values again, we can see that there are no more null values and we can use this dataset for further model development.

```
# Check for null values
null_values = dryBeans_dataset.isnull().sum()
print("Null Values:\n", null_values)
```

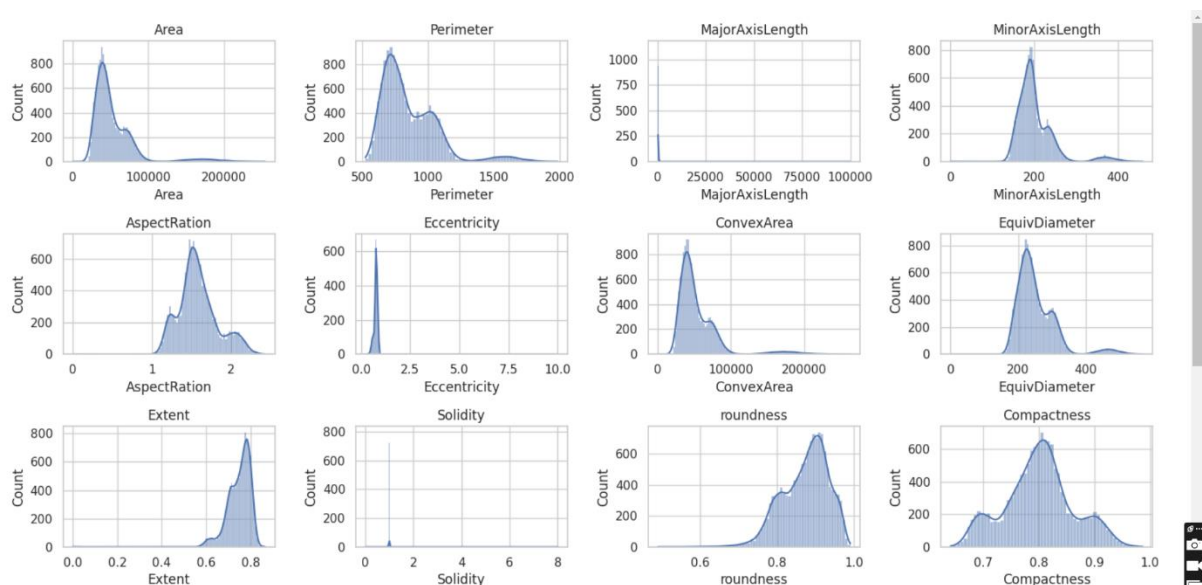
```
Null Values:
Area          0
Perimeter     0
MajorAxisLength  0
MinorAxisLength  0
AspectRatio    0
Eccentricity   0
ConvexArea     0
EquivDiameter  0
Extent         0
Solidity       0
roundness      0
Compactness    0
ShapeFactor1   0
ShapeFactor2   0
ShapeFactor3   0
ShapeFactor4   0
Class          0
dtype: int64
```

Exploratory data analysis:

The features in the Dry Bean dataset are derived from the shape characteristics of individual beans. It's probable that the original data comprised images of the beans, which were subsequently analyzed to extract numerical features representing different aspects of the bean's shape.

Here's a breakdown of some features and what they might represent in terms of bean shape:

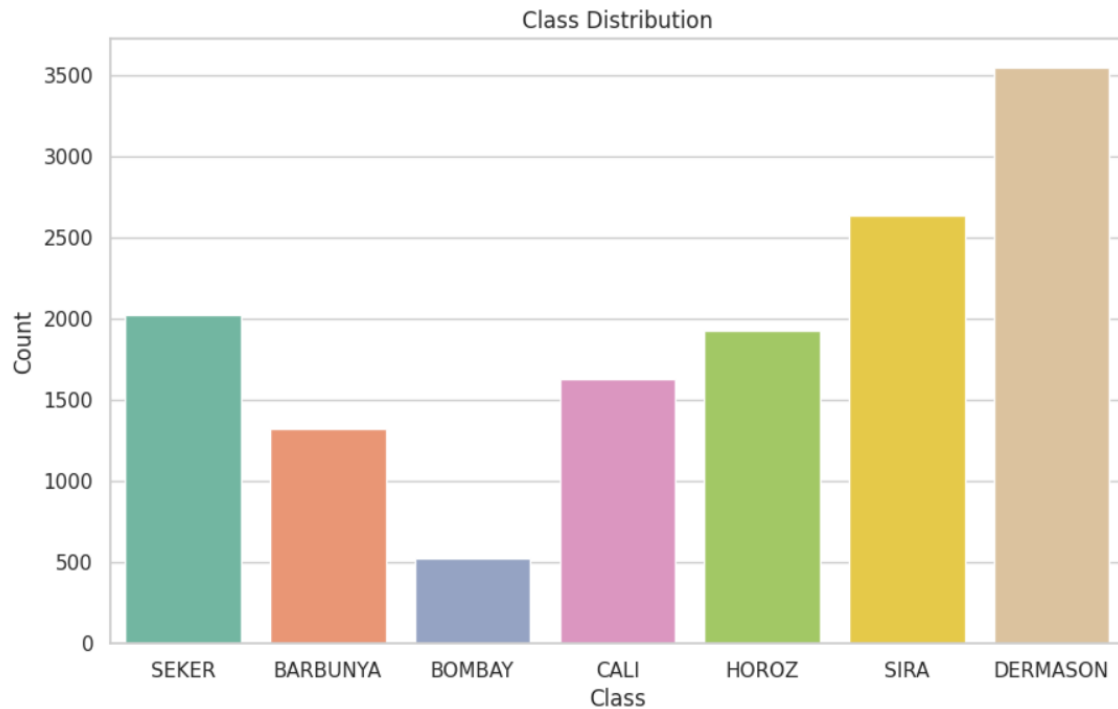
- Area, Perimeter: These capture the overall size and outline of the bean.
- Major Axis Length, Minor Axis Length: These indicate the longest and shortest dimension of the bean, respectively. The difference between these values helps determine how elongated the bean is.
- Aspect Ratio: This is the ratio of the major axis length to the minor axis length. A value closer to 1 suggests a rounder bean, while values far from 1 indicate a more elongated shape.
- Eccentricity: This measures how elliptical a shape is. A perfect circle would have an eccentricity of 0, while higher values indicate increasing elongation.
- Convex Area: This represents the area of the smallest convex shape (like a polygon) that completely encloses the bean.



The above histograms provide numerical insights into the dataset's bean shape features:

1. Size Metrics (Area, Perimeter, Major Axis Length, Minor Axis Length): These features exhibit similar distributions, with a peak around 500 and a tail extending to higher values, suggesting a predominance of smaller beans with some size variation.
2. Aspect Ratio: The wider distribution indicates a mix of bean shapes, with values ranging from close to 0 to 1. Low aspect ratios denote elongated beans, while high ratios suggest more circular shapes.
3. Eccentricity: The histogram shows a peak around 0.5, with values ranging from close to 0 to 1. Most beans exhibit some degree of ellipticity, indicating shape variation.
4. Convex Area and Equivalent Diameter: These features display similar distributions to size metrics, indicating variation in bean size.
5. Extent: Most beans have an extent peak around 0.8, suggesting they are well-filled, with some variability in filling level.

6. Solidity: The solidity histogram peaks around 0.9, indicating most beans are fairly solid, though some variability exists in solidity.



The barplot shows us the number of beans in each class of beans. The BOMBAY class beans seems to have the least number of beans in the dataset, whereas DERMASON has the highest count of beans in the dry beans dataset.

Model development:

State the hyperparameters selected for the models and how/why you selected those hyperparameters.

LDA:

We're separating the dataset in two parts: features (X) and target variable (Y). 'X' drops the column 'Class' and has all the other columns. 'y' holds the target variable class in it.

Model Development

```
✓ [11] # Separate features (X) and target variable (y)
0s X = dryBeans_dataset.drop(columns=['Class'])
    y = dryBeans_dataset['Class']
```

After parting our dataset into two ways, the X and y. We now import 'train_test_split' function from the sklearn.model_selection library from python.

X_train will have training features whereas the X_test will have testing features in it. Similarly with the target variable (y). The attribute test_size determines the proportion of data to be allocated for testing and training. In most of the cases, 20% of data will be used for testing and the remaining 80% for training purposes. Last attribute is optional but, it is a random number generator to shuffle the data before splitting.

The standard scaler is used to standardize features by scaling to unit variance and removing the mean of the data. The .fit part of the standardscaler is used to calculate the mean and standard deviation of all the features. It is essential to use the fit() method on training data only rather than other half of the separated data. The .transform(X_train) transforms the training feature using mean and standard deviation. This step applies the scaling transformation to the testing features (X_test) using the same scaler object (scaler) that was previously fitted to the training data. By using the same scaler object, we ensure consistency in the scaling process, utilizing the means and standard deviations calculated during the fitting process on the training data. The resulting scaled testing features are stored in X_test_scaled.

```
✓ [12] from sklearn.model_selection import train_test_split
0s
    # Split the dataset into training and testing sets (e.g., 80% train, 20% test)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
✓ [13] from sklearn.preprocessing import StandardScaler
0s
    # Initialize StandardScaler
    scaler = StandardScaler()

    # Fit and transform the training set
    X_train_scaled = scaler.fit_transform(X_train)

    # Transform the testing set
    X_test_scaled = scaler.transform(X_test)
```

The first line indicates that an instance with LDA() has been created this model is used for dimensionality reduction and classification of tasks. 'store_covariance' is an attribute that specifies that it should store covariance matrices for each class. The 'lda.fit()' trains the model with two parameters - "X_train_scaled" is the training data that has been standardized and scaled using the standardscaler() and "y_train" is a variable containing the class labels for each data point in the

training set, serving as the target for training. `Lda_predictions = lda.predict(X_test_scaled)` line is used to predict the class labels

```
✓ [14] # Initialize and fit LDA model
0s   lda = LDA(store_covariance=True)
      lda.fit(X_train_scaled, y_train)

      # Make predictions on the testing set
      lda_predictions = lda.predict(X_test_scaled)

      # Create labels based on the predictions
      labels = lda_predictions
```

Decision Trees:

- The code begins by importing essential libraries needed for various tasks such as data splitting, model evaluation, and preprocessing.
- Next, it divides the dataset into feature variables (X) and the target variable (y), with X containing all columns except the 'Class' column, which signifies the target variable.
- Normalization is performed using the MinMaxScaler from scikit-learn to scale the numerical features within a specified range, typically between 0 and 1, ensuring consistent scaling across all features.
- Subsequently, the code selects the columns intended for scaling, which include all columns except the 'Class' column. The scaler is then fitted to these selected columns, determining the minimum and maximum values for each feature used in the scaling process.
- Finally, the data undergoes transformation using the fitted scaler, effectively applying the scaling transformation to normalize the numerical features.

```
▶ from sklearn.model_selection import train_test_split
  from sklearn.metrics import accuracy_score
  from sklearn.preprocessing import MinMaxScaler

  # Split the dataset into training and testing sets
  X = drybean.drop('Class', axis=1)
  y = drybean['Class']

  # Normalize numerical values into ranges
  scaler = MinMaxScaler()

  # Select the columns you want to scale
  columns_to_scale = drybean.columns[:-1]

  # Fit the scaler on the selected columns
  scaler.fit(drybean[columns_to_scale])

  # Transform the selected columns using the scaler
  drybean[columns_to_scale] = scaler.transform(drybean[columns_to_scale])
```

- We split our resampled data into two parts: one for training the model (80%) and the other for testing it (20%).
- Then, we created a Decision Tree Classifier and trained it using the training data. After that, we visualized the decision tree

using a function called `plot_tree`, which helps us understand how the model makes decisions. Finally, we displayed the plot of the decision tree.

```
from sklearn.model_selection import cross_val_score
import numpy as np

# Define a range of values for hyperparameters (e.g., max_depth)
param_range = [1, 5, 10, 15, 20]

# Initialize lists to store mean cross-validation scores and standard deviations
mean_scores = []
std_scores = []

# Iterate over each hyperparameter value
for param in param_range:
    # Initialize and train the decision tree classifier with the current hyperparameters
    clf = DecisionTreeClassifier(max_depth=param)

    # Perform cross-validation and compute scores
    scores = cross_val_score(clf, X_res, y_res, cv=5)

    # Compute mean and standard deviation of scores
    mean_score = np.mean(scores)
    std_score = np.std(scores)

    # Append mean score and standard deviation to lists
    mean_scores.append(mean_score)
    std_scores.append(std_score)

# Plot the cross-validation error rate graph
plt.figure(figsize=(10, 6))
plt.errorbar(param_range, mean_scores, yerr=std_scores, fmt='-o', color='b')
plt.xlabel('Max Depth')
plt.ylabel('Cross-Validation Accuracy')
plt.title('Cross-Validation Error Rate')
plt.grid(True)
plt.show()
```

- The code begins by defining a list **param_range** that includes various values for the hyperparameter to be optimized, which in this case is **max_depth** for the decision tree classifier.
- It then initializes two empty lists, **mean_scores** and **std_scores**, to store the average cross-validation scores and their standard deviations, correspondingly. Following this, the code loops through each value in the **param_range** list.
- Within this loop, a decision tree classifier (referred to as **clf**) is instantiated with the current **max_depth** value. Subsequently, cross-validation is executed utilizing **cross_val_score**, evaluating the model's performance through k-fold cross-validation (with **cv=5**).

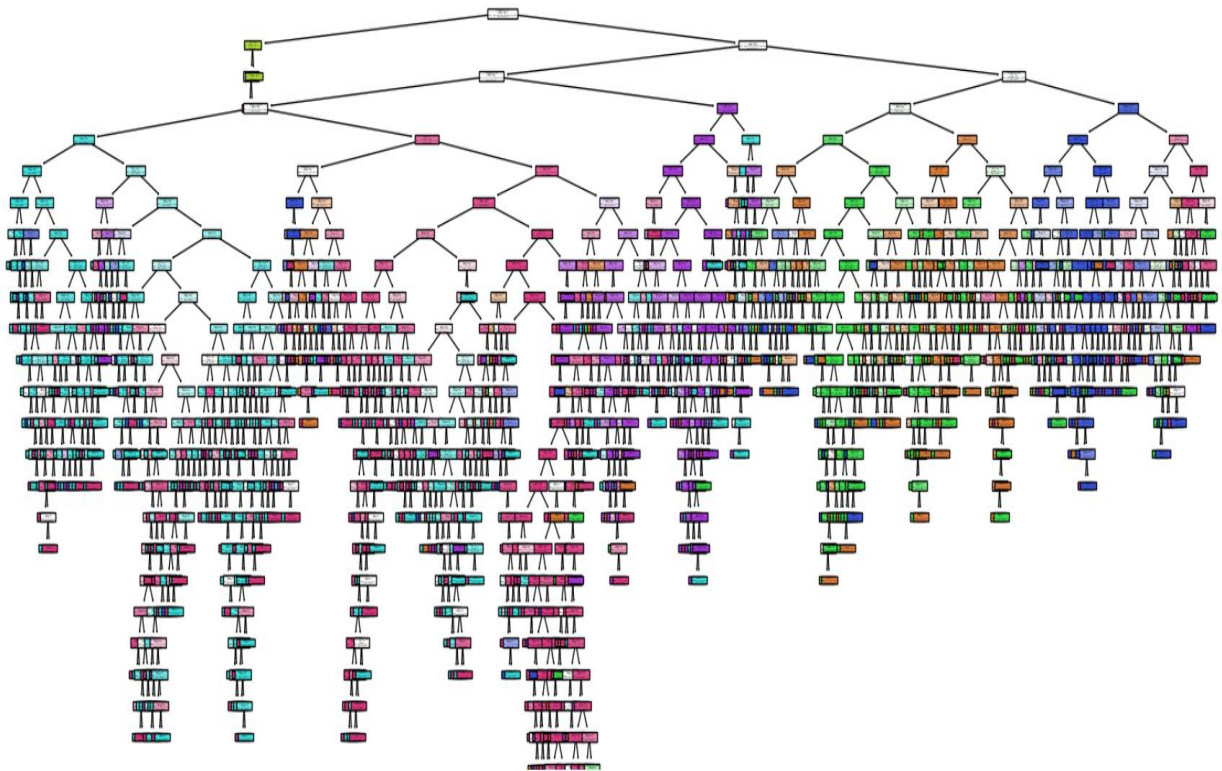
- The average cross-validation score and standard deviation are then calculated and added to the **mean_scores** and **std_scores** lists, respectively. Finally, the code generates a plot illustrating the cross-validation error rate using **matplotlib**.
- The x-axis displays the **max_depth** values, while the y-axis represents the mean cross-validation accuracy. Error bars are incorporated to visualize the variability of the cross-validation scores.

```
[8] from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Split the resampled dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_size=0.2, random_state=42)

# Train the decision tree classifier
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

# Plot the decision tree
plt.figure(figsize=(20,10))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['DERMASON', 'SIRA', 'SEKER', 'HORUZ', 'CALI', 'BARBUNYA', 'BOMBAY'])
plt.show()
```



- The dataset is initially divided into feature variables (`x_data`) and the target variable (`y`). Subsequently, the data undergoes further partitioning into training and testing subsets using the `train_test_split` function. 80% of the dataset is allocated for training (`X_train, y_train`), while the remaining 20% is designated for testing (`X_test, y_test`).
- Following the data splitting process, a Decision Tree model (`dt_model`) is instantiated and trained utilizing the `fit` method with the training data (`X_train, y_train`).
- Once the model has been trained, it is utilized to generate predictions on the test set (`X_test`), resulting in predicted values (`y_pred`) that are subsequently compared with the actual values (`y_test`). The accuracy of the model is then computed using the `accuracy_score` function.
- In addition to evaluating the model's performance on the test set, cross-validation is conducted via the `cross_val_score` function utilizing 5-fold cross-validation. This process involves assessing the model's accuracy across various data subsets to provide a more reliable estimation of its performance. Finally, the average accuracy across all folds is computed and displayed as the cross-validated accuracy

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score

# Split the dataset into training and testing sets
x_data = drybean.drop('Class', axis=1)
y = drybean['Class']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(x_data, y, test_size=0.2, random_state=42)

# Train a Decision Tree model
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)

# Evaluate the model on the test set
y_pred = dt_model.predict(X_test)

# Evaluate the accuracy of the Decision Tree model
accuracy = accuracy_score(y_test, y_pred)
formatted_accuracy = format(accuracy, '.0%')
print("Accuracy (without cross-validation):", formatted_accuracy)

X_merged_df = X_train.append(X_test)
y_merged_df = y_train.append(y_test)

# Perform cross-validation
cv_scores = cross_val_score(dt_model, X_merged_df, y_merged_df, cv=5) # Use 5-fold cross-validation
average_accuracy = cv_scores.mean()
formatted_cv_accuracy = format(average_accuracy, '.0%')
print("Cross-validated Accuracy:", formatted_cv_accuracy)
```

Accuracy (without cross-validation): 89%
Cross-validated Accuracy: 89%

```
[2] # Calculate the count of each target variable
count = drybean['Class'].value_counts()
print(count)
```

```
DERMASON    3546
SIRA         2636
SEKER        2027
HOROS        1928
CALI         1630
BARBUNYA     1322
BOMBAY        522
Name: Class, dtype: int64
```

Data preprocessing entails examining our dataset and addressing any null values present, which could impede further analysis of the dataset. Calculate the count of each target variable.

K-Means Clustering:

Hyper-parameters:

"n_clusters": The number of clusters that will form (eight is the default).

"n_init": the total number of times (by default, 10) that the K-means algorithm will be executed using various centroid seeds

Selection reason:

This defines how many clusters K-Means will use to split the data. The number of unique classes that are contained in the 'Class' variable of your data is cleverly used by the code to determine how many clusters there are. This is expecting that the number of classes and natural clusters in the data are equal.

Also, it is standard procedure to run the K-Means method 10 times with various random initializations to maximize the probability of finding better cluster centers and avoid getting stuck in local minimums. For this reason, n_init=10 is probably set.

✓ Unsupervised methods (K-Means, Hierarchical Clustering)

K-means by Sindhu

```
0s from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
import seaborn as sns
from sklearn.metrics import confusion_matrix
from sklearn.metrics import silhouette_score
```

```
0s from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.cluster.hierarchy import fcluster
```



```

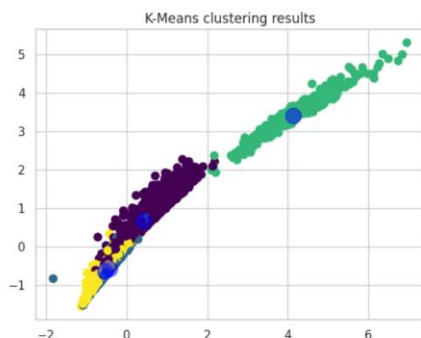
✓ 2s ▶ kmeans = KMeans(n_clusters=4, random_state=48)
kmeans.fit(X_train_scaled)

# Get cluster centers and labels
centers = kmeans.cluster_centers_
labels = kmeans.labels_

# Visualize clustering results (select two d Loading...
plt.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='Blue', s=200, alpha=0.5)
plt.title("K-Means clustering results")
plt.show()

# Displaying the dataset with cluster labels
dryBeans_dataset.head(10)

```



	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRatio	Eccentricity	ConvexArea	EquivDiameter	Extent	Solidity	roundness	Compactness	ShapeFactor1	ShapeFactor2	ShapeFactor3	Shape
0	28395	610.291	208.178117	173.888747	1.197191	0.549812	28715.0	190.141097	0.763923	0.988856	0.958027	0.913358	0.007332	0.003147	0.834222	(
1	28734	638.018	200.524796	182.734419	1.097356	0.411785	29172.0	191.272751	0.783968	0.984986	0.887034	0.953861	0.006979	0.003564	0.909851	(
2	29380	624.110	212.826130	175.931143	1.209713	0.562727	29690.0	193.410904	0.778113	0.989559	0.947849	0.908774	0.007244	0.003048	0.825871	(
3	30008	645.884	210.557999	182.516516	1.153638	0.498616	30724.0	195.467062	0.782681	0.976696	0.903936	0.928329	0.007017	0.003215	0.861794	(
4	30140	620.134	201.847882	190.279279	1.060798	0.333680	30417.0	195.896503	0.773098	0.990893	0.984877	0.970516	0.006697	0.003665	0.941900	(
5	30279	634.927	212.560556	181.510182	1.171067	0.520401	30600.0	196.347702	0.775688	0.989510	0.943852	0.923726	0.007020	0.003153	0.853270	(
6	30477	670.033	211.050155	184.039050	1.146768	0.489478	30970.0	196.988633	0.762402	0.984081	0.853080	0.933374	0.006925	0.003242	0.871186	(
7	30519	629.727	212.996755	182.737204	1.165591	0.513760	30847.0	197.124320	0.770682	0.989367	0.967109	0.925480	0.006979	0.003158	0.856514	(
8	30685	635.681	213.534145	183.157146	1.165852	0.514081	31044.0	197.659696	0.771561	0.988436	0.954240	0.925658	0.006959	0.003152	0.856844	(
9	30834	631.934	217.227813	180.897469	1.200834	0.553642	31120.0	198.139012	0.783683	0.990810	0.970278	0.912125	0.007045	0.003008	0.831973	(

```

✓ 6s ▶ # Calculate silhouette scores for different number of clusters
for n_clusters in range(2, 6):
    kmeans = KMeans(n_clusters=n_clusters, random_state=48)
    cluster_labels = kmeans.fit_predict(X_train_scaled)
    silhouette_avg = silhouette_score(X_train_scaled, cluster_labels)
    print("For n_clusters =", n_clusters,
          "The average silhouette_score is :", silhouette_avg)

```

```

[ ] /usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `r
warnings.warn(
For n_clusters = 2 The average silhouette_score is : 0.3897576228885212
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `r
warnings.warn(
For n_clusters = 3 The average silhouette_score is : 0.3984238079435358
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `r
warnings.warn(
For n_clusters = 4 The average silhouette_score is : 0.34826897182616856
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `r
warnings.warn(
For n_clusters = 5 The average silhouette_score is : 0.3763220690062463

```

Model development – Hierarchical Clustering:

Initialization: The hierarchical clustering model (ag) should be created first. To set up the model's data grouping, provide parameters like n_clusters, and linkage.

Training: Train the model by fitting X using fit() method.

Matrix Computation: Calculate c_linkage to get the relationship among the clusters. In the code we used complete and single linkage.

Visualisation: Dendrograms have been used to visualize the clusters.

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
method = 'agglom-complete'

ag = AgglomerativeClustering(n_clusters=7, linkage='complete', compute_full_tree=True)
ag.fit(X)

dryBeans_dataset[method] = ag.labels_

c_linkage = linkage(ag.children_, method='complete') # Compute the linkage using the complete method
fig, ax = plt.subplots(1, 1, figsize=(8, 8)) # Show the dendrogram
dendrogram(c_linkage, ax=ax)
plt.show()
```

```
[13] from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
method = 'agglom-single'

ag = AgglomerativeClustering(n_clusters=7, linkage='single', compute_full_tree=True)
ag.fit(X)

dryBeans_dataset[method] = ag.labels_

c_linkage = linkage(ag.children_, method='centroid') # Compute the linkage using the complete method
fig, ax = plt.subplots(1, 1, figsize=(8, 8)) # Show the dendrogram
dendrogram(c_linkage, ax=ax)
plt.show()
```

Performance evaluation:

LDA:

The code below indicates calculating the p-value and trying to understand which predictor is statistically significant to the target variable. Upon fetching results, I see only 3 predictors being statistically significant to the response variable and those are Roundness, Compactness, and ShapeFactor3 with each p-value less than 0.5 which is a common threshold for significance.

✓
0s

```
from scipy.stats import norm

# Get the coefficients and intercept from the LDA model
coefficients = lda.coef_
intercept = lda.intercept_

# Calculate the p-values for each predictor
p_values = 2 * norm.cdf(-abs(coefficients), loc=0, scale=1)

# Print the p-values for each predictor
print("P-values for each predictor:")
for i, feature in enumerate(X.columns):
    print(f"{feature}: {p_values[0, i]}")
```

➡ P-values for each predictor:
Area: 3.906303387783422e-216
Perimeter: 2.0711556107597045e-126
MajorAxisLength: 0.9717252955786121
MinorAxisLength: 0.23583554736716406
AspectRatio: 0.13090084792783413
Eccentricity: 0.8960276870159132
ConvexArea: 4.8837636231227177e-08
EquivDiameter: 8.880995204829977e-241
Extent: 0.7260306388087068
Solidity: 0.994026232892248
roundness: 0.0005392263708830548
Compactness: 0.0
ShapeFactor1: 3.3265189610759263e-28
ShapeFactor2: 0.9935562603684228
ShapeFactor3: 0.0
ShapeFactor4: 0.16782017274924532

✓ Performance Evaluation

```
✓ 1s ▶ from mlxtend.plotting import plot_confusion_matrix

# Evaluate predictions
from sklearn.metrics import confusion_matrix, classification_report

# Get predictions from the LDA model
lda_predictions = lda.predict(X_test_scaled)

# Calculate confusion matrix and classification report
confusion = confusion_matrix(y_test, lda_predictions)
classification_rep = classification_report(y_test, lda_predictions)

print("Confusion Matrix:")
print(confusion)
print("\nClassification Report:")
print(classification_rep)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
plot_confusion_matrix(confusion, cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Confusion Matrix:

The confusion matrix provides a table that shows the number of correct and incorrect predictions made by your LDA model. Here's a breakdown of the table entries in the image you sent (considering the labels are BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER, SIRA):

- Rows: Represent the true labels (actual class labels in the data).
- Columns: Represent the predicted labels (what the model predicted for each class).

Values on the diagonal (e.g., 220 in the top-left corner): Represent the number of correct predictions for each class. In this case, 220 "BARBUNYA" beans were correctly classified as "BARBUNYA" by the model.

Values off the diagonal (e.g., 22 in the top-left corner): Represent the number of incorrect predictions. For example, 22 beans that were actually "CALI" were incorrectly predicted as "BARBUNYA" by the model.

Key Observations from Confusion Matrix:

- Overall Accuracy: The accuracy of the model can be roughly estimated by summing the correct predictions on the diagonal and dividing by the total number of predictions (all entries in the confusion matrix). A higher value on the diagonal indicates better overall performance.
- Class-wise Performance: Analyze how well the model performed for each class by looking at the corresponding rows. Ideally, you want most values to be on the diagonal, indicating correct predictions for that class.
- Common Misclassifications: Identify frequent errors by looking at the high values off the diagonal. For example, if a particular class has a high value in a specific column, it implies the model often confuses that class with the class represented by the column label.

Classification Report:

The classification report provides a more detailed evaluation of the model's performance for each class. Here's an explanation of the metrics included in the report you provided (assuming the labels are the same as the confusion matrix):

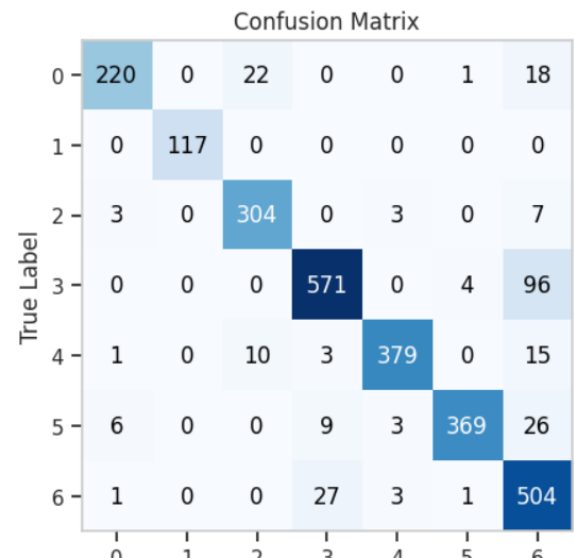
- Precision: This metric represents the proportion of predicted positives (e.g., predicted "CALI" beans) that were actually correct (true positives). A high precision for a class indicates that the model is good at identifying that class without many false positives.
- Recall: This metric represents the proportion of actual positives (e.g., actual "CALI" beans in the data) that were correctly identified by the model (true positives). A high recall for a class indicates that the model is good at capturing most of the instances of that class.
- F1-score: This metric is a harmonic mean of precision and recall, combining both metrics into a single score. A high F1-score for a class indicates a good balance between precision and recall.
- Support: This represents the total number of data points (beans) in that class according to the true labels.

Confusion Matrix:
[[220 0 22 0 0 1 18]
[0 117 0 0 0 0 0]
[3 0 304 0 3 0 7]
[0 0 0 571 0 4 96]
[1 0 10 3 379 0 15]
[6 0 0 9 3 369 26]
[1 0 0 27 3 1 504]]

Classification Report:

	precision	recall	f1-score	support
BARBUNYA	0.95	0.84	0.89	261
BOMBAY	1.00	1.00	1.00	117
CALI	0.90	0.96	0.93	317
DERMASON	0.94	0.85	0.89	671
HOROZ	0.98	0.93	0.95	408
SEKER	0.98	0.89	0.94	413
SIRA	0.76	0.94	0.84	536
accuracy			0.90	2723
macro avg	0.93	0.92	0.92	2723
weighted avg	0.91	0.90	0.91	2723

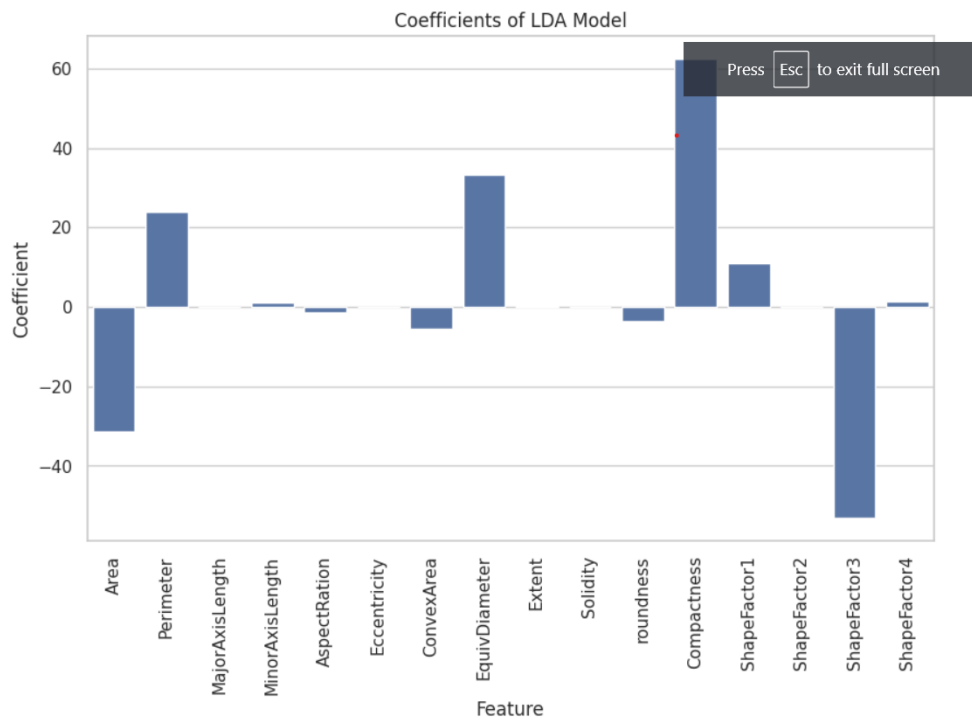
<Figure size 800x600 with 0 Axes>



✓
0s

```
import seaborn as sns
import matplotlib.pyplot as plt

# Plot coefficients
plt.figure(figsize=(10, 6))
sns.barplot(x=X.columns, y=lda.coef_[0])
plt.xticks(rotation=90)
plt.title('Coefficients of LDA Model')
plt.xlabel('Feature')
plt.ylabel('Coefficient')
plt.show()
```



✓
0s

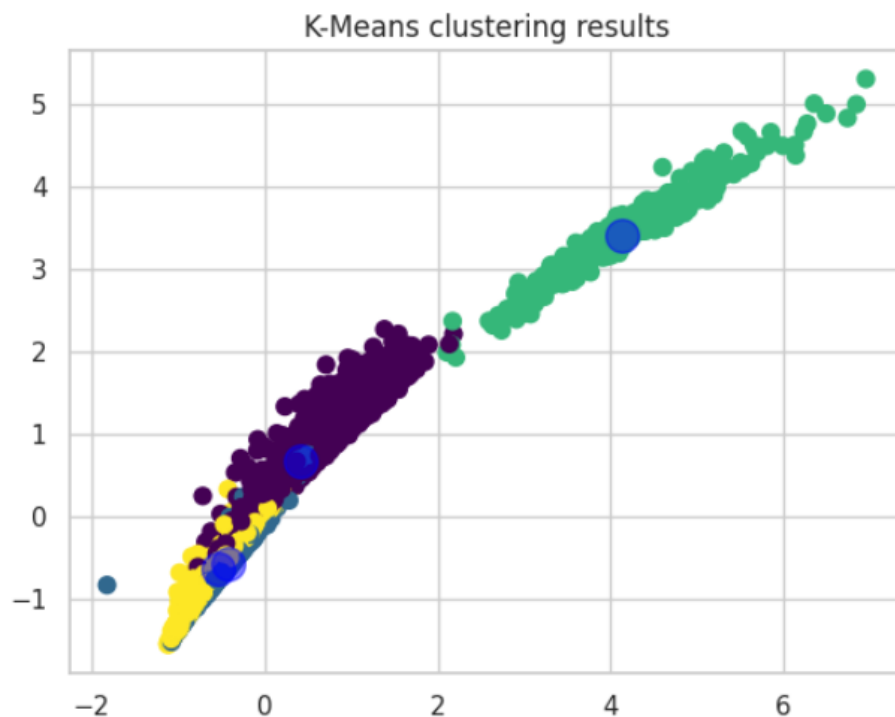
```
from sklearn.metrics import accuracy_score

# Get predictions from the LDA model
lda_predictions = lda.predict(X_test_scaled)

# Calculate accuracy
accuracy = accuracy_score(y_test, lda_predictions)
print("Accuracy:", accuracy)
```

➡ Accuracy: 0.9048843187660668

K Means:



```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `n_init`  
warnings.warn(  
For n_clusters = 2 The average silhouette_score is : 0.3897576228885212  
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `n_init`  
warnings.warn(  
For n_clusters = 3 The average silhouette_score is : 0.3984238079435358  
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `n_init`  
warnings.warn(  
For n_clusters = 4 The average silhouette_score is : 0.34826897182616856  
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `n_init`  
warnings.warn(  
For n_clusters = 5 The average silhouette_score is : 0.3763220690062463
```

Decision Trees:

The data is initially divided into features (`x_data`) and the target variable (`y`), and then further split into training and testing sets using the `train_test_split` function.

The training set constitutes 80% of the data, while the remaining 20% forms the test set. A Decision Tree classifier (`dt_model`) is trained on the training data using the `fit` method. Subsequently, the model's accuracy is evaluated on the test set by comparing its predictions (`y_pred`) with the actual labels (`y_test`) using the `accuracy_score` function.

The resulting accuracy score is then formatted and displayed. Additionally, cross-validation is performed using 5-fold cross-validation (cv=5) to assess the model's performance across different subsets of the data. The mean accuracy across all folds is computed and presented to provide a more reliable estimate of the model's accuracy.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score

# Split the dataset into training and testing sets
x_data = drybean.drop('Class', axis=1)
y = drybean['Class']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(x_data, y, test_size=0.2, random_state=42)

# Train a Decision Tree model
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)

# Evaluate the model on the test set
y_pred = dt_model.predict(X_test)

# Evaluate the accuracy of the Decision Tree model
accuracy = accuracy_score(y_test, y_pred)
formatted_accuracy = format(accuracy, '.0%')
print("Accuracy (without cross-validation):", formatted_accuracy)

X_merged_df = X_train.append(X_test)
y_merged_df = y_train.append(y_test)

# Perform cross-validation
cv_scores = cross_val_score(dt_model, X_merged_df, y_merged_df, cv=5) # Use 5-fold cross-validation
average_accuracy = cv_scores.mean()
formatted_cv_accuracy = format(average_accuracy, '.0%')
print("Cross-validated Accuracy:", formatted_cv_accuracy)-
```

```
Accuracy (without cross-validation): 89%
Cross-validated Accuracy: 89%
```

```

from sklearn.model_selection import cross_val_score
import numpy as np

# Define a range of values for hyperparameters (e.g., max_depth)
param_range = [1, 5, 10, 15, 20]

# Initialize lists to store mean cross-validation scores and standard deviations
mean_scores = []
std_scores = []

# Iterate over each hyperparameter value
for param in param_range:
    # Initialize and train the decision tree classifier with the current hyperparameters
    clf = DecisionTreeClassifier(max_depth=param)

    # Perform cross-validation and compute scores
    scores = cross_val_score(clf, X_res, y_res, cv=5)

    # Compute mean and standard deviation of scores
    mean_score = np.mean(scores)
    std_score = np.std(scores)

    # Append mean score and standard deviation to lists
    mean_scores.append(mean_score)
    std_scores.append(std_score)

# Plot the cross-validation error rate graph
plt.figure(figsize=(10, 6))
plt.errorbar(param_range, mean_scores, yerr=std_scores, fmt='-o', color='b')
plt.xlabel('Max Depth')
plt.ylabel('Cross-Validation Accuracy')
plt.title('Cross-Validation Error Rate')
plt.grid(True)
plt.show()

```

The above code indicates cross validation error rate

The param_range list encompasses various values for the hyperparameter under scrutiny, namely max_depth, which dictates the maximum depth of the decision tree.

Two empty lists, mean_scores and std_scores, are initialized to capture the mean cross-validation scores and their respective standard deviations.

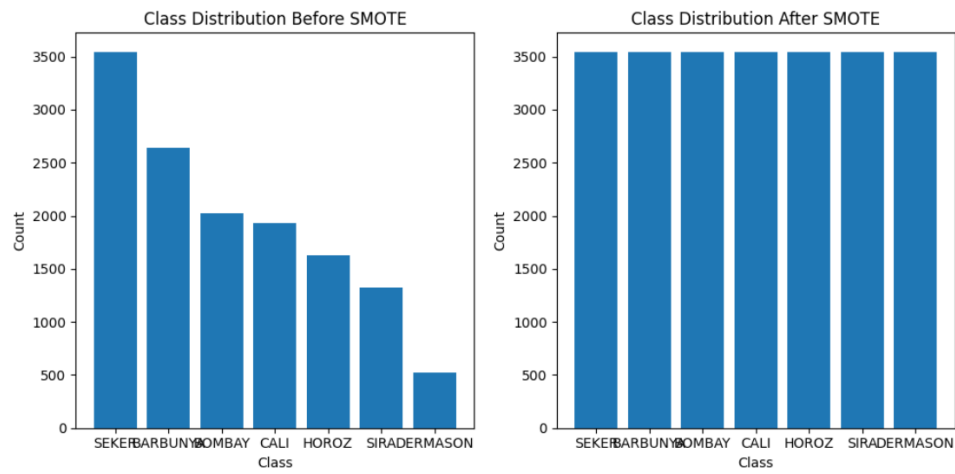
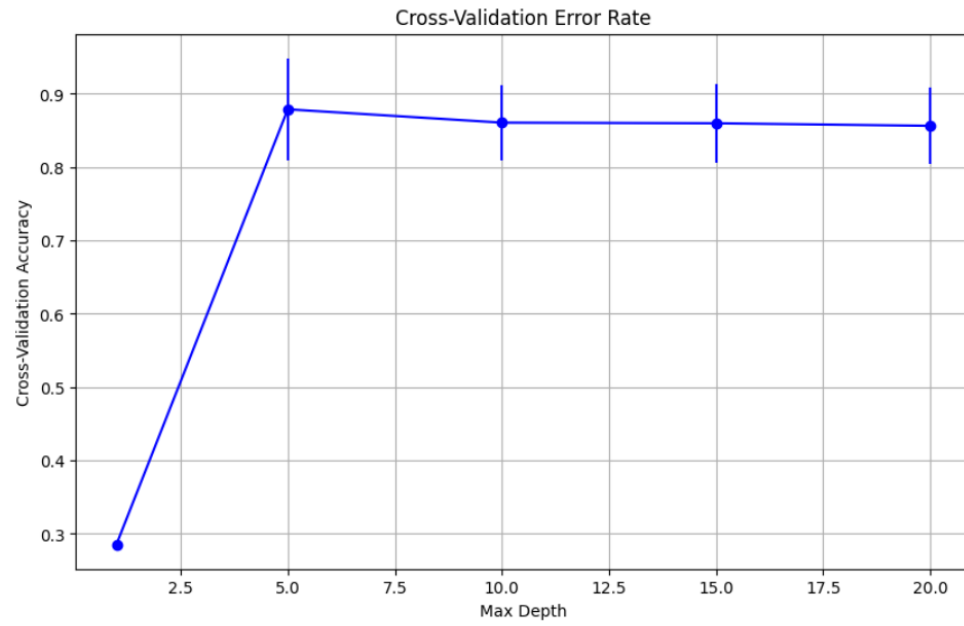
Through a loop iterating over each value in the param_range list, a decision tree classifier (clf) is instantiated with the prevailing max_depth setting.

Cross-validation is then executed utilizing the cross_val_score function with 5-fold cross-validation (cv=5), thereby appraising the model's efficacy through cross-validation and generating an array of scores.

Following this, the mean and standard deviation of the cross-validation scores are computed employing NumPy's np.mean and np.std functions.

Subsequently, the resultant mean cross-validation score and standard deviation are appended to the mean_scores and std_scores lists, correspondingly.

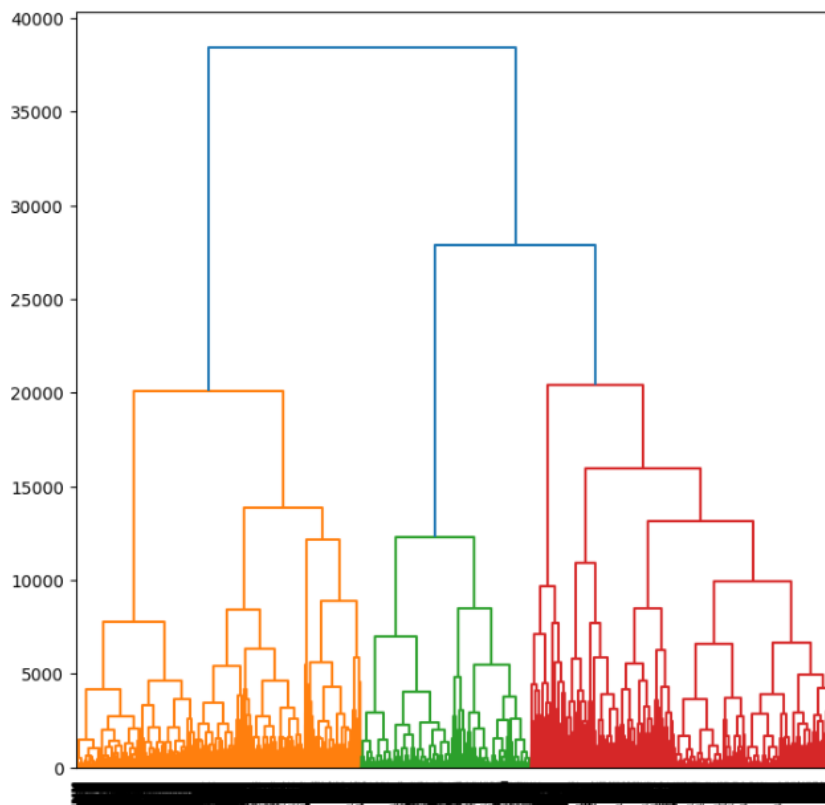
Ultimately, the code plots the graph of the cross-validation error rate utilizing matplotlib, with the max_depth hyperparameter values along the x-axis and the mean cross-validation accuracy on the y-axis. Incorporation of error bars aids in visualizing the variance in the cross-validation scores.

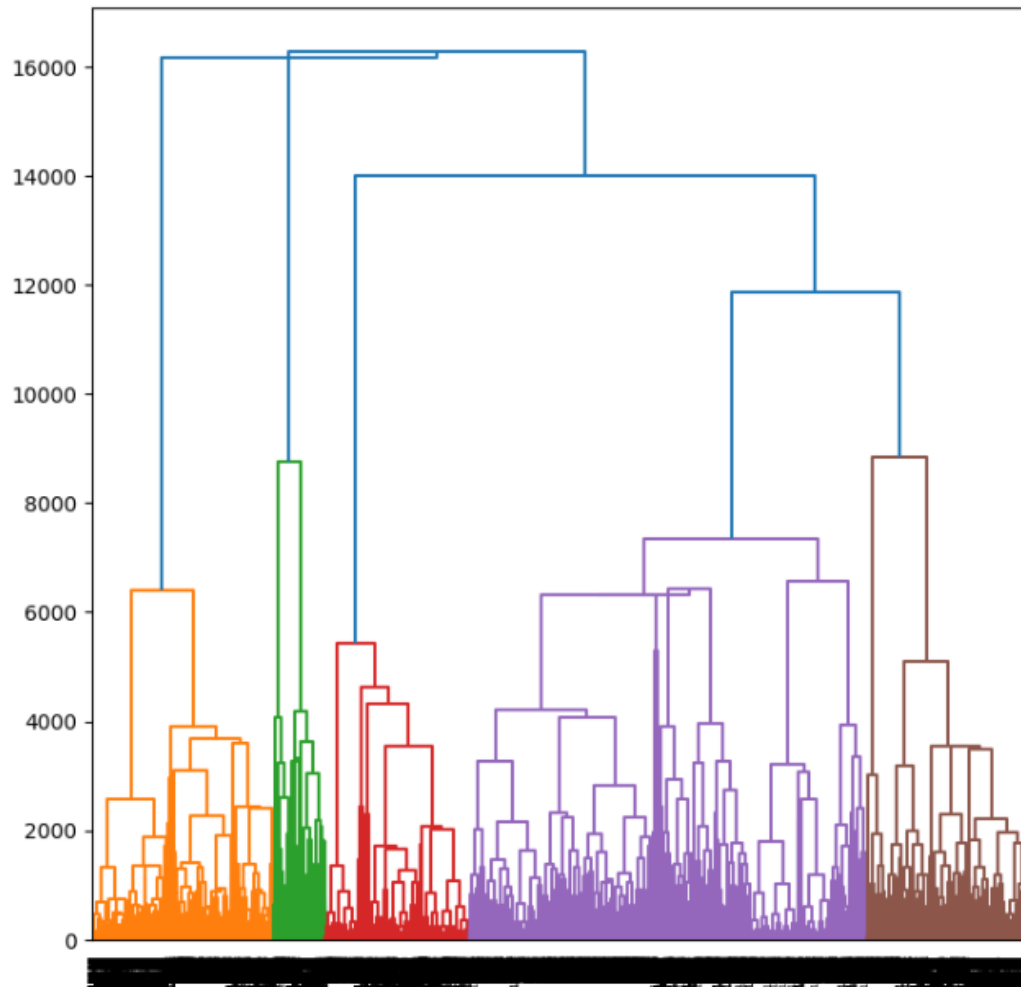


Hierarchical Clustering:

In Hierarchical Clustering we are using complete and single linkage. The dendrogram plots the hierarchy of the clusters formed by the algorithm. The complete linkage produces compact, similar size clusters. It shows that it is less sensitive to outliers.

Whereas, in single linkage it considers the minimum distance between two points. Single linkage produced elongated, irregularly shaped cluster. Single linkage works better on large dataset.





Interpretation:

K-Means:

After PCA transformation, KMeans clustering is performed on the data. A predefined number of clusters is created by grouping data points together using an unsupervised learning technique called KMeans. The program is instructed to create six clusters in this case. Every data point is assigned to the cluster with the nearest centroid based on the features present in the reduced dimension space. In the end, a scatter plot is made to display the clustered data. Each data point is plotted, and the values along the first two principal components (PC1 and PC2) determine where each data point appears on the plot. The visual identification of clusters is facilitated by the display of data points that belong to the same cluster in the same color. Recognizing the quality of the data points

Hierarchical Clustering:

An alternative to k-means clustering is hierarchical clustering, which eliminates the requirement to predetermine the number of groups. * Graphs showing the hierarchical links within the data are produced by hierarchical clustering, as opposed to k-means, which separates data into discrete groups. Hierarchical clustering, which is typically based on an agglomerative approach, begins with individual data points to form clusters, which are thereafter progressively merged to form a structure like a tree from leaves to roots. This method has an advantage over k-means since it can determine clusters more flexibly, can be visually interpreted through dendrograms, and has a hierarchical

structure. These features are especially useful when deciding the number of clusters or when the underlying relationships in the data are complex.

LDA: it has an accuracy of 0.904 which is not a worse performance of the supervised model. However, it is **greater** while compared to the decision trees **as it is 89%**.

Conclusions:

In this project, we explored a variety of unsupervised and supervised methods for analyzing and clustering data. Our objective was to understand the strengths and limitations of each method and gain insights into the underlying structures of the dataset.

Unsupervised Methods:

1. LDA

LDA is a probabilistic generative model commonly used for topic modeling.

Through LDA, we identified latent topics within our dataset, allowing us to uncover hidden thematic structures.

The interpretability of topics provided valuable insights into the main themes present in the data.

LDA proved effective for understanding the underlying content distribution but required careful parameter tuning.

2. Decision Trees:

Decision Trees are a versatile supervised learning method for classification and regression tasks.

By constructing decision trees, we were able to visualize the decision-making process and identify important features for classification.

Decision Trees provided interpretable models that could be easily understood by non-technical stakeholders.

However, Decision Trees are prone to overfitting, especially with complex datasets, and may require pruning to generalize well to unseen data.

Supervised Methods:

1. K-means Clustering:

K-means clustering is a popular unsupervised learning algorithm for partitioning data into K clusters.

Through K-means clustering, we identified distinct clusters within the dataset based on similarity metrics.

The simplicity and scalability of K-means made it suitable for large datasets with known or estimated numbers of clusters.

However, the choice of the number of clusters (K) and the sensitivity to initial cluster centroids are important considerations.

2. Hierarchical Clustering:

Hierarchical clustering builds a hierarchy of clusters, providing insights into both fine-grained and coarse-grained structures.

By visualizing dendrograms, we gained a deeper understanding of the hierarchical relationships within the data.

Hierarchical clustering does not require specifying the number of clusters beforehand, offering flexibility in cluster identification.

However, hierarchical clustering can be computationally intensive, especially for large datasets, and may not scale well.

Conclusion:

In conclusion, our exploration of unsupervised and supervised methods provided valuable insights into the structure and patterns within our dataset. LDA and Decision Trees offered interpretable models for understanding content distribution and classification, respectively. Meanwhile, K-means and Hierarchical Clustering provided clustering solutions for identifying meaningful groupings within the data. The choice of method ultimately depends on the specific objectives of the analysis, the nature of the data, and the interpretability requirements.

By leveraging a combination of these methods, we can gain a comprehensive understanding of complex datasets and extract actionable insights to inform decision-making processes.

This conclusion summarizes the key findings and implications of employing unsupervised methods like LDA and Decision Trees, as well as supervised methods like K-means clustering and Hierarchical clustering, providing a holistic view of the data analysis process.